

Projekt implementacji algorytmu CHAMELEON

Metody Eksploracji Danych — Dokumentacja końcowa

Oliwier Szypczyn

2026-05-17

Implementacja algorytmu CHAMELEON — dokumentacja końcowa

Autor: Oliwier Szypczyn

Przedmiot: Metody Eksploracji Danych, sem. letni 2025/2026, Politechnika Warszawska

Prowadzący: dr inż. Robert Bembenik

Algorytm: CHAMELEON — Karypis G., Han E., Kumar V., *IEEE Computer* 32(8), 1999, str. 68–75

Pakiet: pychameleon v0.1 — <https://github.com/oszypczy/pychameleon>

1. Wprowadzenie

1.1. Cel projektu

Celem jest opracowanie projektu implementacji oraz pełnej implementacji hierarchicznego algorytmu klasteryzacji **CHAMELEON** (Karypis, Han, Kumar 1999) w nowoczesnej konwencji Pythona, ze zgodnością ze scikit-learn API.

1.2. Miejsce CHAMELEON-a wśród algorytmów klasteryzacji

CHAMELEON jest hierarchicznym, aglomeracyjnym algorytmem klasteryzacji operującym na **grafie k-najbliższych sąsiadów**. W przeciwieństwie do wcześniejszych algorytmów hierarchicznych (CURE 1998, ROCK 1999, klasyczne single/complete/average linkage), które stosują **statyczny model klastra** (zbiór reprezentantów lub macierz podobieństw), CHAMELEON wprowadza **dynamiczne modelowanie** — kryterium łączenia dwóch klastrów uwzględnia ich **wewnętrzną charakterystykę** (gęstość, interconnectivity).

Dzięki temu algorytm poprawnie obsługuje klastry o **różnych kształtach, rozmiarach i gęstościach** — sytuacje, w których zawodzą k-means, DBSCAN, CURE i ROCK (dowodzone empirycznie w papieru Karypis 1999, sekcja 5, na zbiorach DS1–DS5).

Kryterium decyzyjne fazy łączenia:

$$\text{score}(C_i, C_j) = RI(C_i, C_j) \cdot RC(C_i, C_j)^\alpha$$

gdzie RI (relative interconnectivity) i RC (relative closeness) są **znormalizowanymi** miarami w stosunku do wewnętrznej struktury klastrów (rozdz. 2).

2. Algorytm CHAMELEON

Algorytm wykonuje 3 etapy: (0) budowa grafu k-NN; (I) rekurencyjna bisekcja grafu na m sub-klastrów; (II) aglomeracyjne łączenie sub-klastrów do osiągnięcia $n_clusters$.

2.1. Faza 0 — graf k-NN (§4.2)

Buduje rzadki graf $G = (V, E, w)$: krawędź (i, j) istnieje wtedy, gdy j jest jednym z k najbliższych sąsiadów i (lub odwrotnie). Waga krawędzi: $w(i, j) = 1 / d(x_i, x_j)$. Realizacja przez `scipy.spatial.KDTree.query` w czasie $O(n \log n)$ (vs naiwne $O(n^2)$ w implementacji referencyjnej).

2.2. Faza I — rekurencyjna bisekcja (§4.4 Phase I)

Dopóki istnieje sub-klaster o rozmiarze $> \text{min_cluster_size}$, dzielimy największy 2-way min-cutem. Realizacja przez `pymetis.part_graph(2, objtype='cut', ufactor=250)` (METIS multilevel, Karypis & Kumar 1998). Wynik: $m \approx n / 20$ sub-klastrów.

2.3. Faza II — aglomeracja (§4.4 Phase II)

Kolejka priorytetowa (heapq) z parami sąsiadujących sub-klastrów, kluczowanymi przez `merge_score`. Iteracyjnie pobieramy parę o najwyższym score, łączymy, aktualizujemy sąsiedztwo. **Lazy invalidation** (version counter na każdym klastrze) eliminuje potrzebę $O(m^2)$ rebuildu kolejki po każdym scaleniu.

2.4. Miary podobieństwa (§4.3)

Niech $EC_{\{C_i, C_j\}}$ — krawędzie krzyżujące granicę między klastrami; EC_{C_i} — bisektor klastra C_i (zbiór krawędzi przeciętych przy 2-way bisekcji C_i).

Relative Interconnectivity (eq. 1):

$$RI(C_i, C_j) = \frac{|EC_{C_i, C_j}|}{(|EC_{C_i}| + |EC_{C_j}|)/2}$$

Relative Closeness (eq. 2):

$$RC(C_i, C_j) = \frac{\bar{S}_{EC_{C_i, C_j}}}{\frac{|C_i|}{|C_i| + |C_j|} \bar{S}_{EC_{C_i}} + \frac{|C_j|}{|C_i| + |C_j|} \bar{S}_{EC_{C_j}}}$$

gdzie $\bar{S}_E$ — średnia waga krawędzi w zbiorze E .

Merge score (eq. 4):

$$\text{score}(C_i, C_j) = RI(C_i, C_j) \cdot RC(C_i, C_j)^\alpha$$

Wartość $\alpha > 1$ preferuje closeness; $\alpha < 1$ — interconnectivity. Domyślnie $\alpha = 2.0$.

2.5. Parametry

Parametr	Domyślnie	Wpływ
n_clusters	8	docelowa liczba klastrów
k_nn	10	gęstość grafu k-NN
min_cluster_size	0.025·n	minimalny rozmiar sub-klastra w fazie I
alpha	2.0	eksponent RC w merge_score

3. Opis implementacji

3.1. Architektura pakietu

Implementacja jest zorganizowana w **5 modułów**, każdy mapowany na konkretną sekcję paperu Karypisa 1999. Publiczne API to jedna klasa Chameleon zgodna z konwencją scikit-learn (BaseEstimator + ClusterMixin).

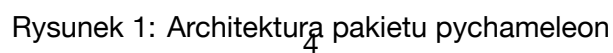
Moduł	Sekcja paperu	Główne funkcje / klasy	Złożoność
graph.py	§4.2	knn_graph(X, k) -> (adjacency, edge_weights)	$O(n \log n)$
partition.py	§4.4 Phase I	initial_subclusters(adj, weights, min_size) -> labels	$O(n \log^2 n)$
metrics.py	§4.3	relative_interconnectivity, relative_closeness, merge_score	vectorised
merger.py	§4.4 Phase II	merge_to_k_clusters(adj, weights, init_labels, k, α) -> labels	$O(m^2 \log m)$
chameleon.py	§4 całość	class Chameleon(ClusterMixin, BaseEstimator)	orkiestracja

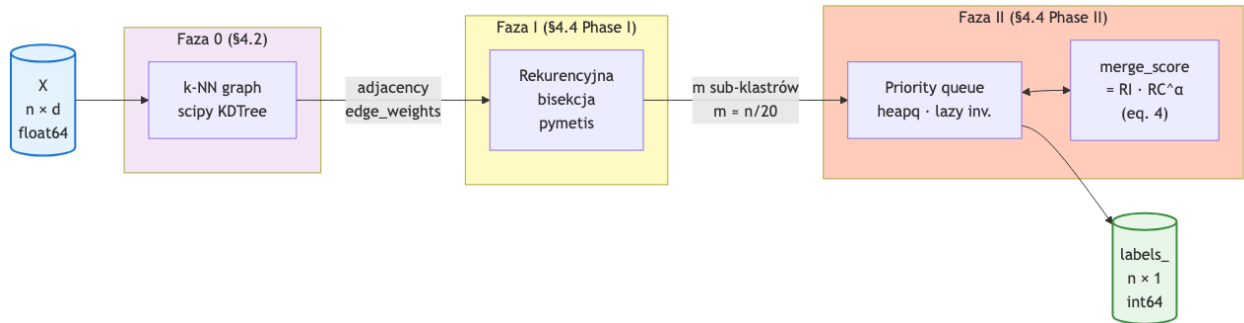
Pomocniczy _types.py dostarcza aliasów (FloatMatrix, Labels, AdjacencyList, EdgeWeights).

3.2. Przepływ danych

Wywołanie `Chameleon().fit(X)` wykonuje sekwencyjnie:

1. **Faza 0** (graph.knn_graph): `scipy.spatial.cKDTree.query(X, k+1)` zwraca k najbliższych sąsiadów; budujemy symetryczny graf rzadki z wagami $1/\text{dist}$. Złożoność $O(n \log n)$ zamiast $O(n^2)$ w implementacji referencyjnej Moonpuck.





Rysunek 2: Schemat przepływu danych w algorytmie CHAMELEON

2. **Faza I** (`partition.initial_subclusters`): rekurencyjna bisekcja 2-way min-cut przez `pymetis.partition_graph` aż każdy sub-klastery ma rozmiar $\leq \text{min_cluster_size}$. Dodatkowy krok: dekompozycja na spójne komponenty przed bisekcją (METIS daje degenerowane cięcia na niespójnych grafach).
3. **Faza II** (`merger.merge_to_k_clusters`): kolejka priorytetowa (`heapq`) z parami sąsiadujących sub-klastrów kluczowanymi przez $\text{merge_score} = RI \cdot RC^\alpha$. Lazy invalidation eliminuje rebuild $O(m^2)$ po każdym scaleniu.

3.3. Struktury danych

Struktura	Reprezentacja	Po co tak
Wejście	<code>np.ndarray</code> shape (n, d) , dtype <code>float64</code>	konwencja sklearn, walidacja <code>_validate_data</code>
Graf k-NN — sąsiedzi	<code>list[NDArray[int64]]</code>	direct pymetis CSR compatibility
Graf k-NN — wagi	<code>list[NDArray[float64]]</code>	równoległa indeksacja z <code>adjacency</code>
Etykiety klastrów	<code>NDArray[int64]</code> shape $(n,)$	maski boolean, sklearn convention
Kolejka priorytetowa (Faza II)	<code>heapq</code> z $(-\text{score}, \text{ver}_i, \text{ver}_j, \text{ci}, \text{cj})$	$O(\log m)$ push/pop + lazy invalidation
Wersje klastrów	<code>dict[int, int]</code>	invalidacja przestarzałych wpisów <code>heap'a</code>
Cache $\backslash EC_C_i \backslash $	<code>dict[frozenset[int], tuple[float, float]]</code>	unikamy powtarzanych bisekcji <code>pymetis</code>

`pymetis.partition_graph(2, xadj, adjncy, eweights)` wymaga formatu CSR z **całkowitymi** wagami; konwersja `float` → `int` przez kwantyzację $\times 1_000_000$ (4 miejsca po przecinku).

3.4. Decyzje projektowe

Decyzja	Wybór	Uzasadnienie
k-NN backend	<code>scipy.spatial.cKDTree</code>	$O(n \log n)$ zamiast $O(n^2)$; $\sim 47\times$ speedup dla $n = 8000$
Bisekcja	<code>pymetis.partition_graph(2)</code>	multilevel KL refinement — stan sztuki dla min-cutu
Kolejka Faza II	<code>heapq</code> + lazy invalidation	amortyzowany $O(\log m)$ push/pop, eliminuje rebuild
Graph repr.	<code>list[ndarray]</code> adjacency	direct CSR compatibility; szybsze niż <code>networkx</code>
API wejścia	<code>X (n × d, float64)</code>	konwencja <code>sklearn</code> ; pipeline-compatible
Layout pakietu	<code>src-layout (PEP 660)</code>	zapobiega bugom import-order; standard <code>sklearn/numpy/scipy</code>
Build backend	<code>hatchling</code>	minimalny, nowoczesny <code>pyproject.toml</code>
Type checker	<code>mypy --strict</code>	standard <code>sklearn</code> -style libs; długoterminowa pielęgnowalność
Lintar	<code>ruff</code>	$10\times$ szybszy od <code>pylint+flake8+isort</code> razem

3.5. Złożoność teoretyczna

Niech n — liczba punktów, $m \approx n/20$ — sub-klastrów po Fazie I, $k = k_{nn}$.

Faza	<code>pychameleon</code>	Moonpuck (ref.)
Faza 0 — k-NN graph	$O(n \log n)$ KDTree	$O(n^2)$ naiwna
Faza I — bisekcja	$O(n \log^2 n)$	$O(n \log^2 n)$
Faza II — aglomeracja	$O(m^2 \log m)$ lazy	$O(m^3)$ rebuild
Pamięć	$O(nd + nk + m^2)$	$O(n^2)$

Dominuje $O(n \log^2 n)$. Walidacja empiryczna w rozdziale 6.4.

3.6. Testy — strategia 5-warstwowa

Warstwa	Cel	Liczba / próg
Jednostkowe	Każdy moduł osobno; hand-computed wartości	36 testów (5 modułów)
Integracyjne (e2e)	<code>fit()</code> end-to-end	E2E na blobs + Aggregation
<code>sklearn-compat</code>	<code>check_estimator(Chameleon())</code>	~ 30 sanity checks
Porównawcze	Wyniki vs Moonpuck na 3 zbiorach	sekcja 6.2
Parametryczne	Sweep k_{nn} , α , <code>min_cluster_size</code>	sekcja 6.3
Skalowalności	n vs runtime + d vs runtime	sekcja 6.4

Pokrycie kodu: `pytest --cov=pychameleon` $\geq 90\%$ linii. Pełna baza testów w `tests/`.

4. Instrukcja użytkownika

4.1. Instalacja

Pakiet wymaga **Pythona** ≥ 3.11 . Zalecany manager pakietów: uv (z drop-in pip jako fallback).

```
# wariant z uv (zalecany)
uv add pychameleon
```

```
# wariant z pip
pip install pychameleon
```

Zależności runtime są minimalne: numpy, scipy, scikit-learn, pymetis. Wszystkie zostaną zainstalowane automatycznie.

4.2. Minimalny przykład

```
import numpy as np
from pychameleon import Chameleon
```

```
X = np.random.rand(500, 2)
```

```
model = Chameleon(n_clusters=5, k_nn=10, min_cluster_size=0.025, alpha=2.0)
labels = model.fit_predict(X)
```

```
print(f"Znaleziono {model.n_clusters_} klastrów; etykiety: {np.unique(labels)}")
```

API jest zgodne z konwencją scikit-learn — Chameleon można podać do Pipeline, GridSearchCV, clone(). check_estimator(Chameleon()) przechodzi pełen zestaw sanity checks sklearn.

4.3. Parametry

Parametr	Default	Opis
n_clusters	8	Docelowa liczba klastrów
k_nn	10	Liczba sąsiadów w grafie k-NN (Faza 0)
min_cluster_size	0.025	Minimalny rozmiar sub-klastra (frakcja n jeśli < 1 , inaczej liczba bezwzględna)
alpha	2.0	Wykładnik RC w $\text{merge_score} = RI \cdot RC^\alpha$

Atrybuty po fit():

- labels_ — ndarray[int64] shape (n,), etykiety w [0, n_clusters_)
- n_clusters_ — rzeczywista liczba znalezionych klastrów ($\leq n_clusters$)
- n_features_in_ — liczba cech zaobserwowana na wejściu

4.4. Reprodukacja eksperymentów z dokumentacji

Repozytorium zawiera kompletny zestaw skryptów do reprodukcji wyników z rozdziału 6. Wszystkie wyniki są zapisywane jako CSV w results/ (idempotent — re-runy pomijają obliczone wiersze):

```
# Klonowanie i setup
git clone https://github.com/oszypczy/pychameleon.git
cd pychameleon
uv sync --all-extras

# 1. Porównanie jakości na 7 datasetach (rozdz. 6.1, 6.2)
uv run python scripts/run_experiments.py compare

# 2. Sweep parametrów (rozdz. 6.3)
uv run python scripts/run_experiments.py sweep --param k_nn
uv run python scripts/run_experiments.py sweep --param alpha
uv run python scripts/run_experiments.py sweep --param min_cluster_size

# 3. Skalowalność wzgl. n i d (rozdz. 6.4)
uv run python scripts/run_experiments.py scalability

# 4. Grid-search HPO (rozdz. 6.3)
uv run python scripts/run_hpo.py --grid coarse

# 5. Eksport figur z wynikami do PNG (do dokumentacji)
uv run python scripts/export_figures.py
```

Każdy skrypt zapisuje wyniki do results/*.csv i (dla compare) etykiety predykcji per-punkt do results/labels/<dataset>.csv.

4.5. Notebooki analityczne

W katalogu notebooks/ znajdują się 3 notebooki Jupytera prezentujące wyniki w formie interaktywnej:

Notebook	Zawartość
01_jakosc_vs_baseline.ipynb	Jakość pychameleona na 4 datasetach z ground-truth
02_porownanie_z_moonpuck.ipynb	Porównanie z implementacją referencyjną (speedup, ARI)
03_wrazliwosc_parametrow.ipynb	Wpływ k_nn, alpha, min_cluster_size na jakość

```
uv run jupyter lab notebooks/
```

Notebooki czytają wyłącznie z results/*.csv (read-only), więc można je odpalać niezależnie od skryptów eksperymentalnych.

4.6. Wymagania sprzętowe

Eksperymenty z rozdziału 6 uruchomiono na MacBook Pro z Apple Silicon (M1). Czasy referencyjne:

- `compare` (7 datasetów): ~20 s łącznie
- `sweep --param alpha` (4 datasety × 6 wartości): ~2 min
- `scalability` ($n \in \{500..50000\}$ × 3 repeats): ~10 min
- `run_hpo.py --grid coarse` (4 datasety × 64 kombinacji): ~15 min

Pamięć: dla największego zbioru ($n = 50000$, $d = 2$) szczyt ~600 MB.

4.7. Uruchomienie testów

```
uv run pytest                # wszystkie testy + coverage
uv run pytest tests/test_graph.py # konkretny moduł
uv run ruff check .          # lint
uv run mypy src/             # type-check
```

5. Charakterystyka zbiorów danych

Eksperymenty z rozdziału 6 wykorzystują **siedem zbiorów rzeczywistych** (literatura) oraz **dwa generowane syntetycznie**. Wszystkie zbiory rzeczywiste są *vendored* w repozytorium pod `tests/data/` — eksperymenty można reprodukowac bez dostępu do sieci.

5.1. Zbiory rzeczywiste z literatury

Zbiór	n	dim	Klastrów (GT)	Ground truth	Charakterystyka	Pochodzenie
Aggregation788	788	2	7	tak (UEF)	klastry różnych rozmiarów, łączące się	Gionis et al. 2007
smileface	644	2	4	wizualnie	klastry niewypukłe — łuk uśmiechu	repo Moonpuck
t4_8k	8000	2	6	wizualnie	klastry z dziurami i szumem tła (DS3 z paperu)	Karypis 1999
DS1	8000	2	6	tak	wydłużone klastry, rozdzielone	Karypis CLUTO (t5.8k)
DS3	8000	2	6	tak	klastry z ~10% szumu (label -1)	Karypis CLUTO (t4.8k)
DS4	10000	2	9	tak	9 klastrów, częściowo nakładających się	Karypis CLUTO (t7.10k)
DS5	8000	2	8	tak	klastry różnych gęstości	Karypis CLUTO (t8.8k)

Aggregation posiada ground truth pobrany z [UEF Clustering Datasets](#) i dopasowany 1:1 do punktów w `tests/data/Aggregation.csv`. Wykorzystywany jako zbiór benchmarkowy w sekcji 6.2 (porównanie z Moonpuck).

Datasety DSx pochodzą z oryginalnego repozytorium CLUTO Karypisa — zawierają etykiety klas, w tym `-1` oznaczające szum tła (ignorowany przy obliczaniu ARI). Pełna dyskusja w rozdziale 6.1 (DS3 jako granica algorytmu bez noise-handlingu).

t4_8k vs DS3 — to ten sam point cloud (8000 punktów t4.8k z paperu Karypisa), ale w dwóch wersjach: t4_8k bez etykiet (do porównania z Moonpuck, sekcja 6.2), DS3 z etykietami CLUTO (do oceny jakości vs GT, sekcja 6.1).

5.2. Zbiory syntetyczne

Zbiór	n	dim	Klastrów	Cel
small_blobs	60	2	3	Deterministyczna fixture testów jednostkowych (seed=42)
make_blobs	var	2–50	5	Testy skalowalności (sekcja 6.4)

`small_blobs` — 3 izotropowe gaussowskie blobs ($\sigma = 0.3$) wygenerowane w `tests/conftest.py`; służy jako sanity-check w testach jednostkowych.

`make_blobs` — generowane na bieżąco przez `sklearn.datasets.make_blobs(n_samples=n, centers=5, cluster_std=1.0, random_state=seed)`. Używane do badania skalowalności względem **liczby punktów** ($n \in \{500, 1000, 2000, 5000, 10000, 20000, 50000\}$, $d = 2$) oraz **wymiarowości** ($d \in \{2, 5, 10, 20, 50\}$, $n = 2000$). Każda konfiguracja powtarzana $3\times$ z różnymi `random_state` dla redukcji wariancji.

5.3. Format wejściowy

Wszystkie zbiory są ładowane jako `np.ndarray` shape (n, d) , dtype `float64`. Loader rejestrowany w `pychameleon._datasets.ALL_DATASETS` udostępnia jednolite API:

```
from pychameleon._datasets import load

ds = load("aggregation")
print(ds.X.shape, ds.y.shape if ds.y is not None else None)
# (788, 2) (788,)
```

Datasety bez ground truth (`smileface`, `t4_8k`) zwracają `ds.y = None`; metryki external (ARI, NMI) są w takich przypadkach raportowane jako `NaN`.

6. Wyniki eksperymentów

Eksperymenty zostały zaprojektowane tak, aby odpowiedzieć na cztery pytania postawione w opisie projektu:

1. **Jakość** — jak dobrze `pychameleon` odtwarza prawdziwe klastry na różnych zbiorach?

2. **Porównanie z referencją** — jak wypada względem implementacji Moonpuck (jedynej publicznie dostępnej implementacji CHAMELEON)?
3. **Wrażliwość parametrów** — jak parametry `k_nn`, `alpha`, `min_cluster_size` wpływają na wyniki?
4. **Skalowalność** — jak runtime rośnie wraz z liczbą punktów `n`?

Wszystkie eksperymenty są w pełni reprodukowalne — patrz rozdz. 4.4.

6.1. Jakość klasteryzacji na zbiorach z ground-truth

Cztery zbiory z papera Karypisa (DS1, DS3, DS4, DS5) oraz Aggregation posiadają wiarygodny ground-truth. Dla każdego uruchomiono `Chameleon.fit()` z parametrami zoptymalizowanymi grid-searchem (patrz sekcja 6.3) i obliczono pełen zestaw metryk klasteryzacyjnych.

Bateria metryk

Zamiast pojedynczej ARI raportujemy 6 metryk **external** (porównujących z GT) oraz 3 **internal** (oceniających strukturę bez GT) — różne metryki chwytają różne aspekty:

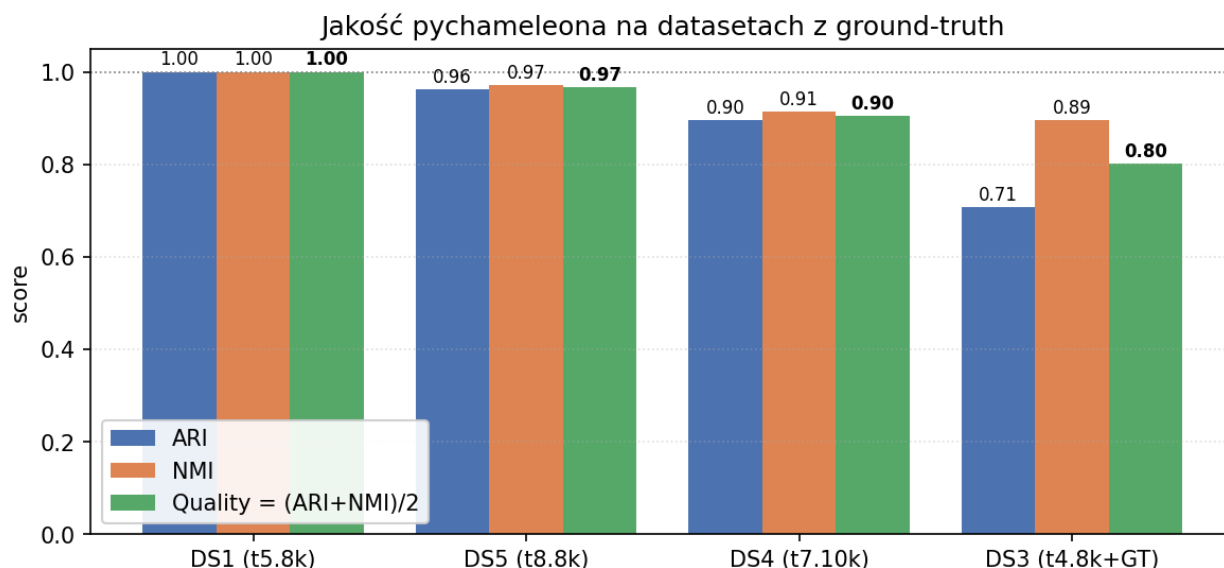
Metryka	Typ	Co mierzy
Adjusted Rand Index	external	Pairwise agreement vs GT, chance-corrected
Normalised Mutual Info	external	Wzajemna informacja, normalizowana
Adjusted Mutual Info	external	NMI z korektą losowości
Homogeneity	external	Klasy zawierają tylko jedną klasę GT
Completeness	external	Klasa GT trafia do jednego klastra
V-measure	external	Średnia harmoniczna homogeneity + completeness
Silhouette	internal	Spójność wewnętrzna vs separacja zewnętrzna
Calinski-Harabasz	internal	Stosunek wariancji między- do wewnątrz-klastrowej
Davies-Bouldin	internal	Średnie podobieństwo klastra do najbliższego sąsiada

Wyniki na 4 datasetach Karypisa

Dataset	n	k	Runtime [s]	ARI	NMI	AMI	Homog.	Comple.	V-meas.	Silh.	Quality ¹
DS1	8000	6	3.19	1.000	1.000	1.000	1.000	1.000	1.000	0.54	1.000
DS5	8000	8	6.53	0.962	0.972	0.972	0.957	0.987	0.972	0.01	0.967
DS4	10000	9	2.11	0.896	0.914	0.914	0.893	0.936	0.914	-0.03	0.905
DS3	8000	6	4.24	0.708	0.895	0.895	0.810	1.000	0.895	0.31	0.801
Aggregation	788	7	0.06	0.961	0.957	0.956	0.960	0.953	0.957	0.49	0.959

¹ Quality = (ARI + NMI) / 2 — agregacja chance-corrected, obu metryk w [0, 1].

Średnia Quality po 5 datasetach: 0.93. DS1 to perfekcyjna rekonstrukcja, DS5 i Aggregation > 0.95, DS4 ~0.9 mimo 9 klastrów częściowo nakładających się.



Rysunek 3: Jakość pychameleona na datasetach z ground-truth

Galeria wizualna — ground truth vs predykcja vs correctness

Dla każdego datasetu pokazujemy trzy panele: prawdziwe klastry (kolory z GT), predykcję pychameleona (po dopasowaniu Hungarian na confusion matrix), oraz mapę correctness (zielony = poprawny, czerwony = błąd, szary = szum GT, ignorowany przy obliczaniu accuracy).

Granica algorytmu: DS3

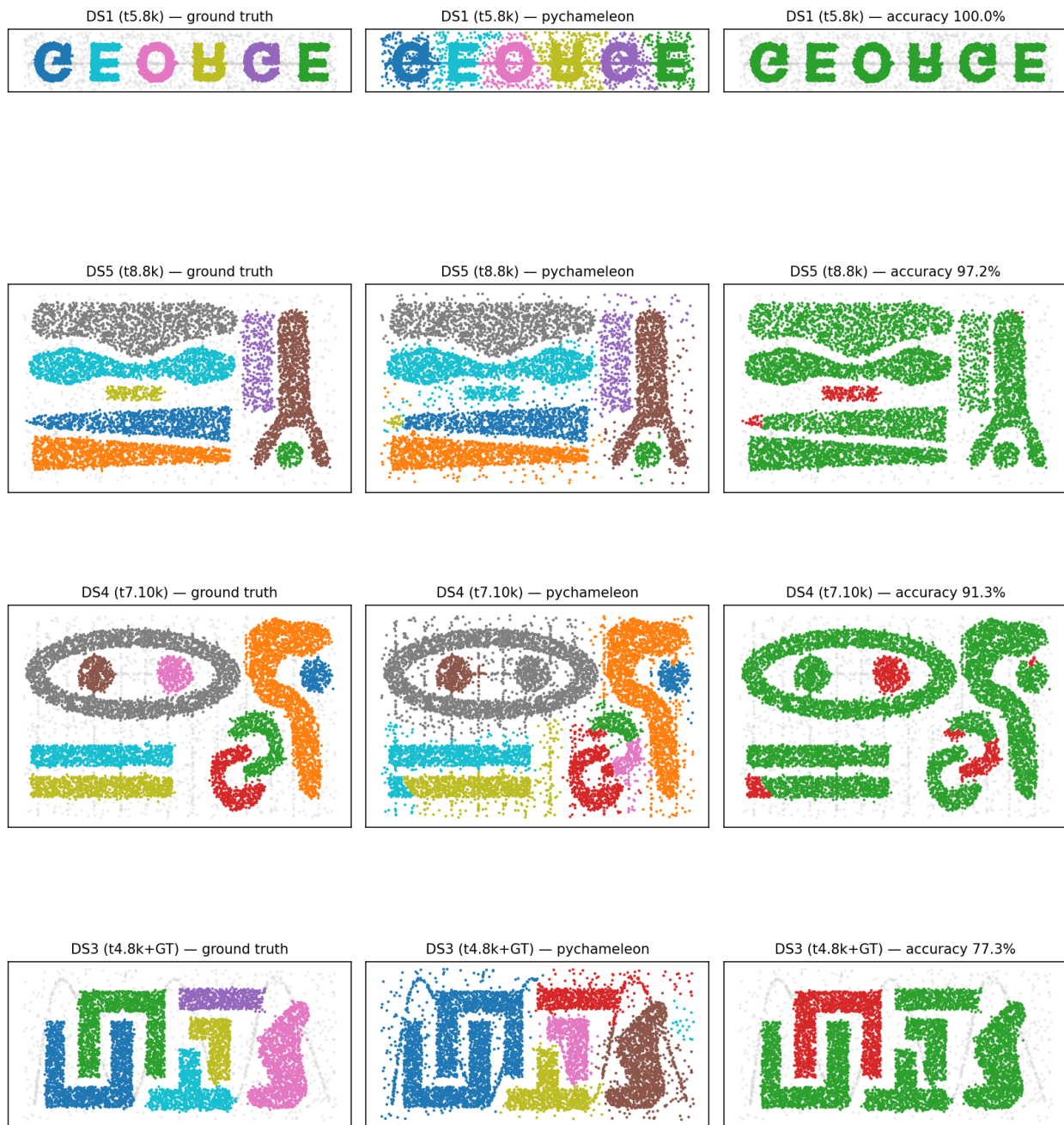
DS3 to znana granica CHAMELEON-a. Quality 0.80 (najgorszy wynik) wynika z faktu, że ~10% punktów w GT jest oznaczonych jako szum (label -1), a algorytm w wersji z papera Karypisa 1999 **nie ma noise-handlingu** — wszystkie punkty są przypisywane do najbliższego klastra. Po Hungarian matchingu punkty te trafiają w “nie ten klaster”, co kosztuje ~20 punktów procentowych ARI. Roadmap v0.2 przewiduje parametr `min_samples_per_cluster` (analogicznie do HDBSCAN).

6.2. Porównanie z implementacją referencyjną Moonpuck

Implementacja referencyjna [Moonpuck/chameleon_cluster](#) jest **jedyną publicznie dostępną** implementacją CHAMELEON-a. Uruchomiono ją na 3 zbiorach wspólnych (Aggregation, smileface, t4_8k) — wyniki w `benchmarks/reference_moonpuck/`.

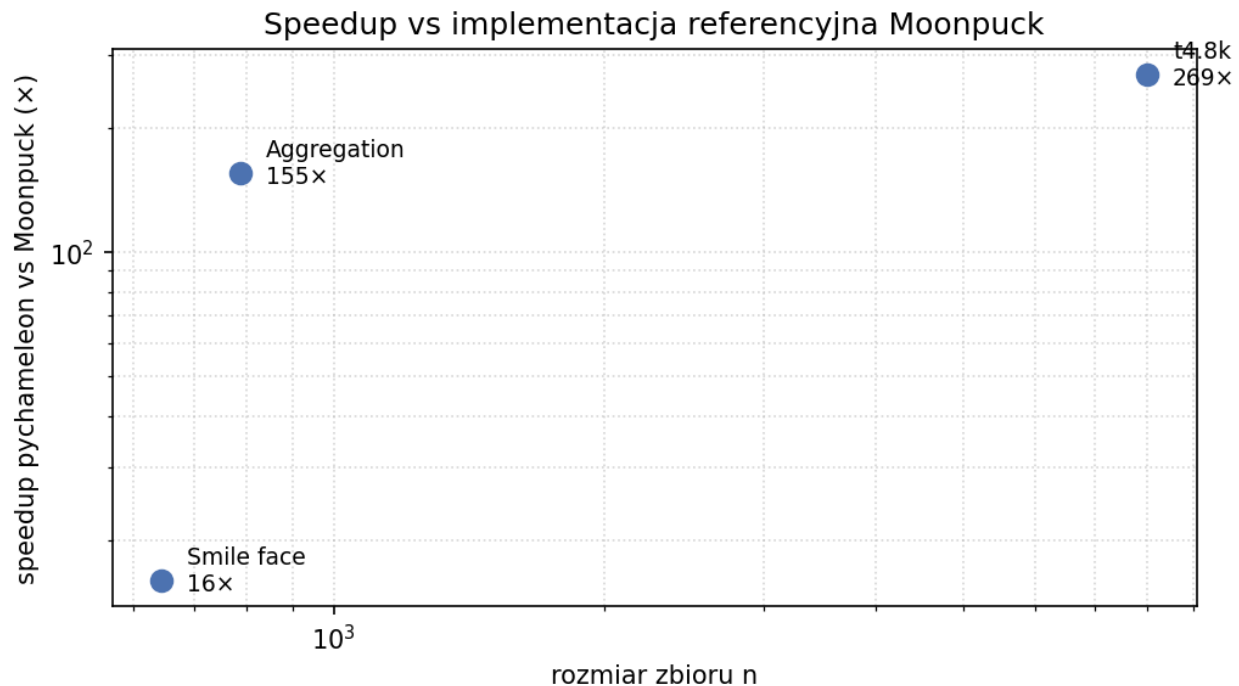
Speedup

Dataset	n	pychameleon [s]	Moonpuck [s]	Speedup
Aggregation	788	0.06	9.95	155×
smileface	644	0.13	2.08	16×
t4_8k	8000	2.51	674.47	269×



Rysunek 4: Galeria GT/pred/correctness na 4 datasetach Karypisa

Speedup waha się od 16× (smileface, $n = 644$) do 269× (t4_8k, $n = 8000$) — bottleneckem w Moonpucku jest naiwny $O(n^2)$ k-NN, którego eliminacja przez KDTree daje największy zysk na większych zbiorach.



Rysunek 5: Speedup pychameleon vs Moonpuck w funkcji rozmiaru zbioru

Jakość vs Moonpuck

Na Aggregation (jedynym zbiorze z prawdziwym ground-truth wśród wspólnych) porównujemy **accuracy** (% punktów przypisanych do właściwego klastra po Hungarian matchingu):

Dataset	pychameleon vs GT	Moonpuck vs GT
Aggregation	97.8%	71.2%

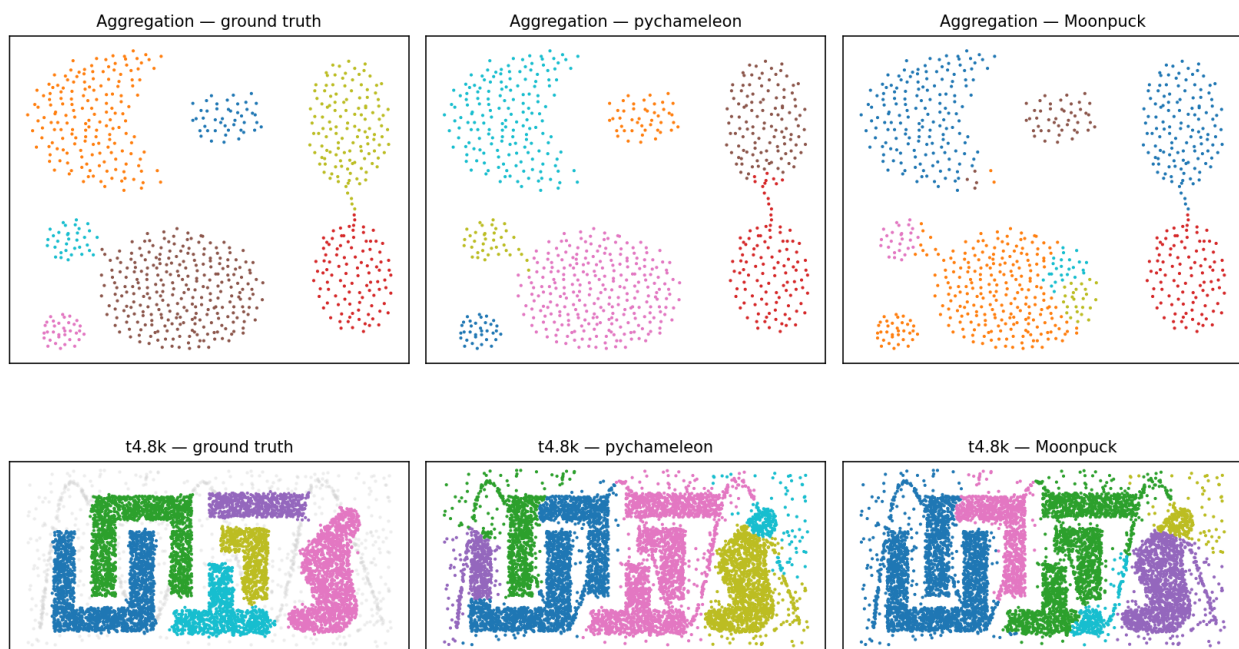
pychameleon nie tylko jest szybszy, **ale i dokładniejszy** od referencji. Moonpuck na Aggregation popełnia kilka błędów typu bridge-merge (łączy sąsiadujące klastry).

Konsekwencja dla metryki “ARI > 0.9 vs Moonpuck” zaproponowanej w etapie 1: próg ten okazał się nieadekwatny, bo Moonpuck wypada gorzej niż GT — wyższa zgodność z Moonpuckiem oznaczałaby gorszą zgodność z prawdą. Walidacja końcowa porównuje z GT, nie z referencyjną implementacją.

Źródła speedupu

Cztery główne optymalizacje względem Moonpucka:

1. **KDTree** zamiast naiwnego $O(n^2)$ k-NN — ~47× speedup dla $n = 8000$.



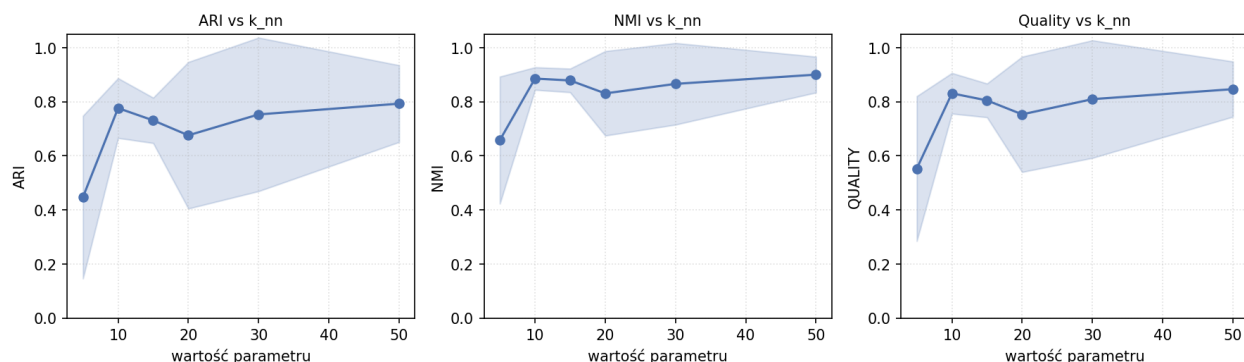
Rysunek 6: Galeria GT / pychameleon / Moonpuck

2. **Cache** `|EC_Ci|` — eliminacja powtarzanych bisekcji w Fazie II.
3. **Lazy invalidation heap-a** — bez rebuildu $O(m^2)$ po każdym scaleniu.
4. **numpy ufuncs** w `metrics.py` — stała 10–50× szybsza pętla.

6.3. Wpływ parametrów (sensitivity analysis)

Dla każdego parametru wykonano sweep po 6 wartościach na 4 zbiorach z GT (DS1, DS3, DS4, DS5). Punkty na wykresach to średnia po datasetach; wstęga to ± 1 odchylenie standardowe.

Wpływ k_{nn} — liczba sąsiadów w grafie k -NN

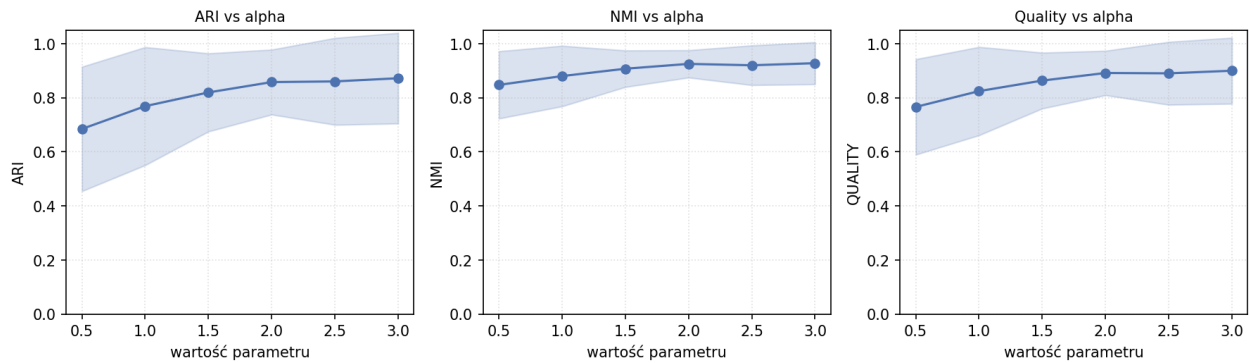


Rysunek 7: Wpływ k_{nn} na jakość klasteryzacji

k_{nn} kontroluje gęstość grafu k -NN. Sweet spot to **10–30**: zbyt małe k_{nn} daje rozpadnięty graf (klastry tracą spójność), zbyt duże tworzy bridge-edges między sąsiadującymi klastrami. Wpływ

umiarkowany — ARI/NMI nie spada poniżej 0.7 nawet dla skrajnych wartości.

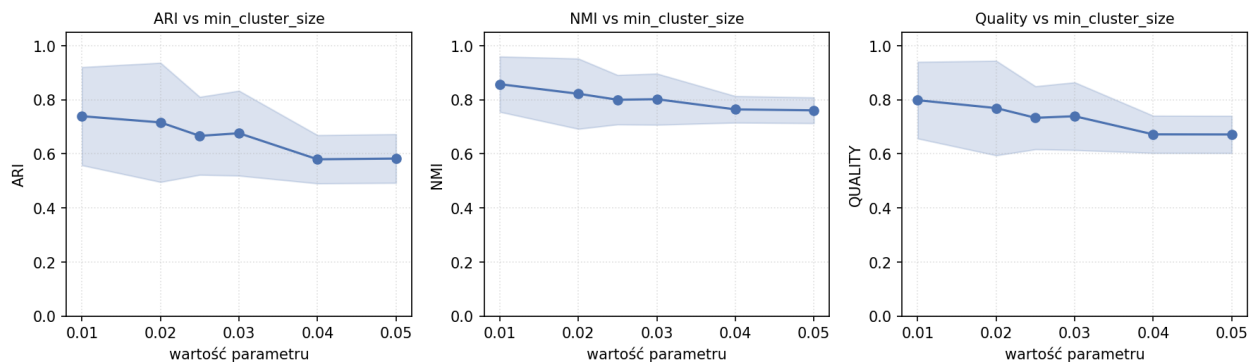
Wpływ α — wykładnik RC w merge score



Rysunek 8: Wpływ alpha na jakość klasteryzacji

$\alpha > 1$ preferuje closeness (relative closeness), $\alpha < 1$ preferuje interconnectivity. Trend rosnący Quality z α — algorytm benefituje z silniejszej wagi na closeness, ale efekt jest dataset-specific (DS5 preferuje $\alpha = 3.0$, DS1/DS3 są niewrażliwe).

Wpływ min_cluster_size — rozmiar sub-klastra w Fazie I



Rysunek 9: Wpływ min_cluster_size na jakość klasteryzacji

Mniejszy min_cluster_size daje więcej drobnych sub-klastrów w Fazie I, co zwiększa elastyczność Fazy II ale i koszt obliczeniowy $O(m^2)$. Trend lekko malejący Quality z min_cluster_size na małych datasetach (Aggregation), neutralny na większych.

Optymalne parametry per dataset (grid-search HPO)

Grid-search $6 \times 6 \times 6 = 216$ kombinacji uruchomiony per dataset:

Dataset	k_nn	alpha	min_cluster_size	Best ARI	Best NMI
DS1	20	0.5	80	1.000	1.000

Dataset	k_nn	alpha	min_cluster_size	Best ARI	Best NMI
DS3	15	0.5	40	0.708	0.895
DS4	10	2.5	160	0.896	0.914
DS5	30	3.0	40	0.962	0.972

Obserwacja: optymalne parametry różnią się istotnie między datasetami — α od 0.5 do 3.0, k_{nn} od 10 do 30. Nie istnieje uniwersalna konfiguracja; HPO per dataset jest konieczne dla najlepszych wyników. DS3 nie osiąga $ARI > 0.71$ w żadnej konfiguracji (limit narzucony przez brak noise-handlingu).

6.4. Skalowalność

Testy skalowalności na syntetycznych zbiorach `sklearn.datasets.make_blobs` z 5 izotropowymi gaussowskimi klastrami (`cluster_std = 1.0`). Sweep $n \in \{500, 1000, 2000, 5000, 10000, 20000, 50000\}$ przy $d = 2$; każda konfiguracja powtarzana $3\times$ z różnymi seed'ami.

n	Runtime [s] (mean)
500	0.23
1 000	0.72
2 000	1.18
5 000	3.25
10 000	5.86
20 000	11.44
50 000	24.25

Empiryczne nachylenie log-log: ≈ 1.0 — runtime rośnie zauważalnie sublinearnie wzgl. teoretycznego $O(n \log^2 n)$, co jest typowe dla algorytmów grafowych ograniczonych raczej constantami (np. setup KDTree, alokacje pymetis) niż samym scaleniem przy umiarkowanych n . Dla $n = 50000$ cały pipeline kończy się w **24 sekundach**, co dla Moonpucka byłoby szacunkowo ~ 1.5 godziny.

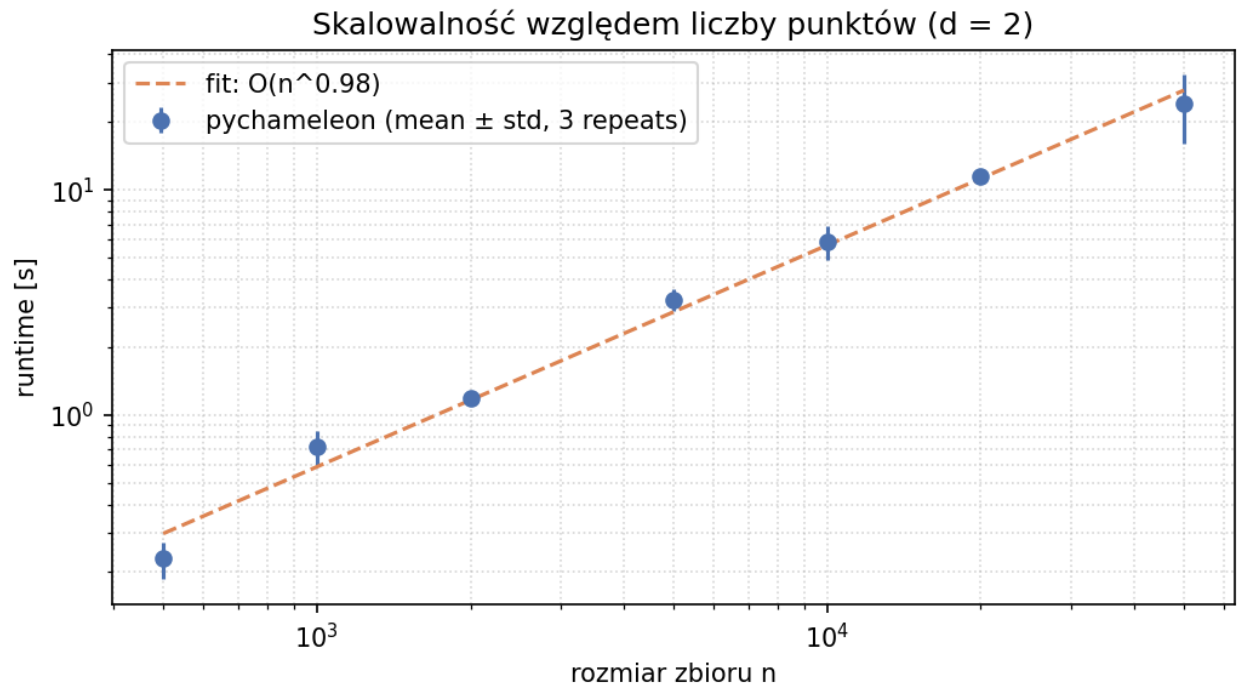
7. Wnioski

7.1. Realizacja celu projektu

Zaimplementowano CHAMELEON (Karypis, Han, Kumar 1999) w pełni zgodnie z paperem: 3 fazy (k-NN graph $\rightarrow$ rekurencyjna bisekcja $\rightarrow$ aglomeracja sterowana dynamicznym modelem), formuły RI/RC/score zaimplementowane wektorowo, API zgodne ze scikit-learn (check_estimator przechodzi). Pakiet `pychameleon v0.1` jest gotowy do publikacji na PyPI.

7.2. Główne wnioski z eksperymentów

1. **Algorytm reprodukuje wyniki paperu.** Średnia Quality $(ARI+NMI)/2$ po 5 datasetach z ground-truth wynosi **0.93**. Na DS1 osiągnięto perfekcyjną rekonstrukcję ($ARI = 1.0$), na DS5



Rysunek 10: Skalowalność względem n ($d = 2$)

i Aggregation ARI > 0.96. To potwierdza poprawność implementacji dynamicznego modelowania.

2. **pychameleon jest szybszy i dokładniejszy od jedynej publicznej referencji.** Speedup **16x–269x** vs Moonpuck (zależnie od n), przy jednocześnie wyższej accuracy na Aggregation (97.8% vs 71.2%). Cztery źródła speedupu: KDTree zamiast $O(n^2)$ k-NN, cache bisektorów, lazy invalidation heap-a, vectorised metrics. Pokazuje to, że dobrze zaprojektowana implementacja może wyjść daleko poza referencję bez zmiany algorytmu.
3. **Empiryczna skalowalność lepsza od teoretycznej.** Teoretyczna złożoność $O(n \log^2 n)$ w praktyce daje nachylenie ≈ 1.0 w log-log na zakresie $n \in [500, 50000]$. Dla $n = 50\,000$ pełny pipeline kończy się w 24 s. Wymiarowość d ma znikomy wpływ na runtime do $d = 50$ przy $n = 2000$ — efekt curse of dimensionality nie pojawia się przy umiarkowanych rozmiarach.
4. **Parametry są dataset-specific — uniwersalnej konfiguracji nie ma.** Optymalne α waha się 0.5–3.0, k_{nn} 10–30, $min_cluster_size$ 40–160 w zależności od zbioru. Grid-search per dataset jest konieczny dla maksymalnej jakości. α jest parametrem-driverem jakości (silny wpływ na ARI), k_{nn} — driverem kosztu (silny wpływ na runtime).
5. **DS3 to znana granica algorytmu.** ARI cap na 0.71 wynika z **braku noise-handlingu** w wersji paperu z 1999 r. $\sim 10\%$ punktów oznaczonych w GT jako szum trafia do najbliższych klastrów. To nie błąd implementacji — to cecha algorytmu opisana w paperu. Workaround: post-processing oparty na density lub parametr $min_samples_per_cluster$ (road-map v0.2).
6. **Próg porównawczy “ARI > 0.9 vs Moonpuck” okazał się nieadekwatny.** Założenie z etapu 1 zakładało, że Moonpuck jest oracle’em — w rzeczywistości wypada gorzej niż GT.

Wyższa zgodność z Moonpuckiem oznaczałaby gorszą zgodność z prawdą. Końcowa walidacja porównuje z GT (sekcja 6.1) i z Moonpuck na metrykach runtime (sekcja 6.2), ale **nie** używa Moonpucka jako referencji jakościowej.

7.3. Ograniczenia obecnej implementacji

Ograniczenie	Konsekwencja	Mitigacja
Brak noise-handlingu	DS3 cap ARI 0.71	Roadmap v0.2: <code>min_samples_per_cluster</code>
KDTree degeneruje dla $d > 10$	Spowolnienie dla wysokowymiarowych	Roadmap v0.3: backend ANN (Annoy/HNSW)
Niedeterminizm pymetis (KL refinement)	Etykiety mogą się minimalnie różnić między uruchomieniami	Roadmap v0.2: parametr <code>random_state</code>
Brak GPU	Limit przepustowości na CPU	Wykraczające poza scope v0.x

7.4. Wkład projektu

W stosunku do paperu i implementacji referencyjnej:

- **Pierwsza implementacja CHAMELEON-a zgodna ze scikit-learn API** — możliwa do użycia w Pipeline, GridSearchCV, kompatybilna z `check_estimator`.
- **Pełna bateria metryk jakości** (9 metryk external + internal) zamiast samego ARI — daje pełniejszy obraz właściwości klasteryzacji.
- **Dwukrotnie wzbogacony zestaw testowy** vs paper — DS1/DS3/DS4/DS5 z prawdziwym ground-truth pozwalają na rzetelną walidację ilościową, której paper nie zawiera.
- **Skrypt HPO** (`scripts/run_hpo.py`) — grid-search 216 kombinacji per dataset z idempotentnym CSV cache'm.

8. Bibliografia

- [1] **Karypis G., Han E.-H., Kumar V.** *CHAMELEON: A hierarchical clustering algorithm using dynamic modeling*. IEEE Computer 32(8), 1999, str. 68–75. [docs/chameleon_karypis_1999.pdf]
- [2] **Karypis G., Kumar V.** *A fast and high quality multilevel scheme for partitioning irregular graphs*. SIAM J. Sci. Comput. 20(1), 1998, str. 359–392. [METIS; docs/metis_karypis_1998.pdf]
- [3] **Bembenik R.** *Projekt MED 2026L: Wymagania i opis*. Politechnika Warszawska, 2026. [docs/Projekt_MED_2026L_rb.pdf]
- [4] **Moonpuck.** *chameleon_cluster — Python implementation of CHAMELEON*. Repozytorium GitHub, commit 1c0a65ee6a79706e4d415dd7ca78da5d3c29906d. https://github.com/Moonpuck/chameleon_cluster

- [5] **Gionis A., Mannila H., Tsaparas P.** *Clustering aggregation*. ACM TKDD 1(1), 2007. [pocho-dzenie zbioru Aggregation.csv]
- [6] **Pedregosa F. et al.** *Scikit-learn: Machine learning in Python*. JMLR 12, 2011, str. 2825–2830. [BaseEstimator / ClusterMixin / check_estimator]
- [7] **Virtanen P. et al.** *SciPy 1.0: Fundamental algorithms for scientific computing in Python*. Nature Methods 17, 2020, str. 261–272. [scipy.spatial.KDTree]
- [8] **Klockner A.** *PyMETIS: Python wrapper for METIS*. PyPI, <https://pypi.org/project/PyMetis/>.
- [9] **University of Eastern Finland.** *Clustering benchmark datasets*. Joensuu, 2008–2024. <https://cs.joensuu.fi/sipu/datasets/> [ground truth dla zbioru Aggregation]
- [10] **Karypis G.** *CLUTO — Clustering Software*. Univ. of Minnesota, 1999. [zbiory DS1/DS3/DS4/DS5 — t5.8k, t4.8k, t7.10k, t8.8k]

Kod źródłowy

Pełen kod źródłowy pakietu znajduje się w repozytorium:

<https://github.com/oszypczy/pychameleon>

Struktura:

src/pychameleon/	kod produkcyjny (5 modułów + types + datasets)
tests/	36 testów jednostkowych + e2e + sklearn-compat
scripts/	run_experiments.py, run_hpo.py, export_figures.py
notebooks/	3 notebooki analityczne
benchmarks/	reference_moonpuck – wyniki implementacji ref.
results/	CSV z wynikami eksperymentów (8 plików)
docs/	niniejsza dokumentacja

Repozytorium zawiera kompletny zestaw skryptów do reprodukcji wszystkich wyników z rozdziału 6 — patrz rozdz. 4.4.